HTML-CSS-05 第5回_まとめ

第5回の目標

- **ハンバーガーメニュー**の作り方を知る
- **positionでの配置**をできるようにする
- **クラス名を意識して**コーディングする

コーポレートサイト

目的

企業や法人に関する情報を総括的に掲載するサイト。企業や法人のことを伝えたり、興味を持ってもらいお問い合わせしてもらうことなどが目的である。

基本構成

- トップページ（ホームページ）
- 詳細ページ
  - サービス
  - 事例
  - 会社概要
- ブログページ（運営しているサービスなどのページ）
- お問い合わせページ

※ 会社の伝えたいことによってページ構成は変動する。

- 会社を信用してよいか確かめるとき、人は公式サイトを見に行く。口コミなどで公式サイトのリンクが使われていると信頼度も上がるので、会社にとってサイトを持つことは大事
- サイトを作るときは「このサイトでクライアントのどんな課題を解決できるか」も考える

実例

- 株式会社メルカリ
- 株式会社TOMAP
- SANKOU!（コーポレートサイト）

コーディングに入る前に：共通スタイル・共通パーツを見つける

コーポレートサイトのような複数ページ・複数セクションのサイトを作るときは、いきなり書き始めず、まず「共通化できるもの」を探すのがポイント。

① CSS変数（カスタムプロパティ）にまとめた方がいい値を見つける

色・コンテンツ幅・z-indexなど、サイト全体で繰り返し使う値は`:root`にまとめて管理する（ CSSカスタムプロパティ（変数）を使ってみよう、第4回も参照）。

```
:root {
  --primary-color: #1863af;
  --accent-color: #fc7f24;
}
```

② layout（レイアウト）でまとめられそうなスタイルを探す

セクションの上下の余白やコンテンツ幅など、ページをまたいで共通する「枠」に関わるスタイルをまとめる。`l_`接頭辞をつけるのが定番（第4回参照）。教材ではスタイルガイド（デザインデータ）が参考になる。

```
.l_container-sm,
.l_container,
.l_container-lg {
  width: 100%;
  padding: 0 16px;
  margin: 0 auto;
}

.l_container-sm {
  max-width: calc(var(--content-width-sm) + 32px);
}

.l_container {
  max-width: calc(var(--content-width) + 32px);
}

.l_container-lg {
  max-width: calc(var(--content-width-lg) + 32px);
}
```

```
/* セクションの上下の余白 */
.l_contents {
  padding: 120px 0;
}
```

クラス	役割
`l_contents`	セクションの上下の余白（今回は上下120pxで統一）
`l_container`	コンテンツ幅（＝セクションの幅のイメージ）

- セクションごとの上下の余白は統一するのが望ましい。バラバラだとサイト全体のまとまりがなくなる
- `l_contents` を用意しておくと、クラス名を呼び出すだけで余白を当てられる。CSSに何度も書かなくて済む
- コンテンツ幅は、PCサイズのときの最大幅（`max-width`）を指定して、画面いっぱいに広がらないようにするもの
- コンテンツ幅を `sm` ／（無印）／ `lg` の3段階で用意し、セクションに合わせて使い分ける
  - `container-sm`：少し狭めのセクション
  - `container`：中間幅のセクション
  - `container-lg`：少し広めのセクション
- `+ 32px` は左右の `padding`（16px × 2）の分（第4回の `l_container` と同じ考え方）。要素が大きくなったとき、中身がpadding分縮んでしまうのを防ぐ
- `l_contents` と `l_container` は実案件でも便利に使える。何をしているか、どこで使われているかを確認しておく

③ module（モジュール）でまとめられそうなスタイルを探す

ハンバーガーメニューやボタンなど、複数箇所で使い回すパーツ。最初から完璧に洗い出す必要はなく、作りながらまとめていけばよい。`m_` 接頭辞をつけるのが定番。

例：使い回すボタン（m_btn）

```
.m_btn {
  width: 100%;
  height: 100%;
  border-radius: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  color: var(--white-color);
```

```
  font-weight: bold;
}

/* 色やアイコンなどの差分はModifierで追加する */
.m_btn__mail {
  background-color: var(--accent-color);
}

.m_btn__mail::after {
  content: "";
  width: 24px;
  height: 24px;
  background: url(../img/btn_email.svg) center / contain no-repeat;
  margin-left: 8px;
}
```

- ボタン本体は`width: 100%; height: 100%;`にしておき、大きさは外側の`.top_kv_btn-wrapper`などの親要素で決める → どこに置いても使い回せる
- `border-radius: 100vh;`：高さに関係なく、両端が必ず半円になる（カプセル型）
- アイコンは`::after`の背景画像として表示している。`contain`で枠内に収め、`no-repeat`で繰り返しを防ぐ

これが第4回で学んだ「クラス名を意識してコーディングする」の実践にあたる。`l_`（レイアウト）・`m_`（モジュール）・ページ名（`top_`など）といった接頭辞を意識して命名していく。

接頭辞のチャート

問い	yes	no
問1：`header`・`footer`・`main`・`container`のどれか？	`l_〇〇`（レイアウト）	問2へ
問2：他のページでも使い回されているか？	`m_〇〇`（モジュール）	問3へ
問3：そのページでしか使わないスタイルか？	`ページ名_〇〇`（ページ独自）	`u_〇〇`（ユーティリティ）

- アクセシビリティの観点から、タグは適切なものを使う（ Webアクセシビリティとは）

ヘッダーをファーストビューに重ねて配置する

コーポレートサイトのトップページでは、ヘッダーをKV（ファーストビュー画像）の上に透過させて重ねるデザインがよくある。 - `position: absolute;` をつけると要素が浮き、下の要素（KV）が上にずれ込んで重なる - `absolute` と `relative` は必ずセットでなくてもよい - `absolute` の要素は、親要素をたどって `relative` などを探す - どこにもなければ、最終的に `body`（画面全体）を基準にする - `body` を基準にしたいときは `relative` を書かなくてよい（今回のヘッダーがこれ） - 考え方：ヘッダーに `position: absolute; top: 0; left: 0;` をかけてKVの上に重ね、 `z-index` で他の要素より前面に表示する（第2回のposition・z-index参照） - ハンバーガーメニューの作り方は後述の「ハンバーガーメニュー」セクション参照

画像に色付きフィルターをかける

`linear-gradient()` を背景画像と重ねて指定することで、疑似要素を使わずに画像へ色フィルターをかけることができる。

```
.sample {
  background: linear-gradient(rgba(51, 51, 51, 0.2), rgba(51, 51, 51, 0.2)),
    url(../img/bg_kv.jpg) center top / cover;
}
```

- `background` プロパティにはカンマ区切りで複数の背景（グラデーションや画像）を重ねて指定できる
- 先に書いたものほど手前に表示されるため、半透明の黒いグラデーションが画像の上に重なり、色フィルターのように見える
- グラデーションの開始色と終了色を同じ色にすることで、フィルターをかけたような見た目になる
- 複数背景で画像の上に半透明のレイヤーを載せる方法

KVを画面いっぱいの高さにする

```
.top_kv {
  height: 100vh;
}
```

- `vh`（viewport height）は画面の高さに対する割合の単位。 `100vh` で、ファーストビューが画面の高さぴったりになる（横幅版は `vw`）
- 実案件では `dvh` もよく使う（ビューポート単位（dvhなど））
  - iOSでは `100vh` がうまく効かない問題があり、 `svh` ・ `lvh` ・ `dvh` という新しい単位で解決できる（iOSの100vh問題とsvh・lvh・dvh）
- 背景画像を表示するには、要素に必ず**高さ**が必要
- CSSのvhやvwの使い方

heightとpaddingで高さを出す違い

方法	特徴	向いているもの
`height`	指定した高さに固定される（中身に関係ない）	高さが決まっているUI（ボタン、ヘッダー）
`padding`（上下）	中身のサイズや量に応じて高さが変わる	可変コンテンツ（テキストリンク、CTAセクションなど）

パーツごとにabsoluteで配置する

- KVの中身は中央、スクロールアイコンはKVの下から32px、のように、パーツごとに`absolute`で配置する
- `100vh`で高さが変わっても、どの画面でも綺麗に表示できる

スクロールを促すアイコン（KVの下部に配置）

```
.top_kv-scroll {
  position: absolute;
  bottom: 40px;
  left: 50%;
  transform: translateX(-50%);
}

.top_kv-scroll_arrow {
  width: 40px;
  height: 40px;
  display: flex;
  justify-content: center;
  align-items: center;
  border-radius: 50%;
  border: 1px solid var(--white-color);
}
```

- `bottom: 40px;`でKVの下端から40pxの位置に置き、`left: 50%; transform: translateX(-50%);`で横方向だけ中央寄せしている（後述の「基準点からのずらし方」参照）
- `border-radius: 50%;`で正方形を正円にできる
- 文字の影は`text-shadow`で付ける（後述）

上下左右中央寄せ

ある要素を、親要素に対して縦横どちらも中央に配置する代表的な方法。

```
.sample-inner {
  position: relative;
}

.sample_box {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
}
```

- `top: 50%; left: 50%;` だけでは、要素自身の左上の角が親要素の中央に来るだけで、要素全体は中央に来ない
- `transform: translate(-50%, -50%);` で、要素自身の幅・高さの半分だけ左上方向にずらすことで、要素の中心が親要素の中心と一致し、正確に上下左右中央寄せになる
- 定番の書き方なのでスニペット登録しておくとよい
- `translate` だけで位置を調整するのはNG。`top`・`left` などは基準の位置そのものを動かし、`transform` は見た目だけを動かす。「`top` などで配置して、`transform` で調整する」と考える
    - `transform` だけだと基準がずれ、ブラウザやデバイスによって誤差が出ることがある
- translate・rotate・scaleを独立したプロパティで書く（発展）

margin: 0 auto; と flexの中央寄せの使い分け

書き方	中央にするもの
`margin: 0 auto;`	**その要素自身**を、親要素の中で左右中央にする
`display: flex;` ＋ `justify-content: center;` ＋ `align-items: center;`	**その要素の中にいる子要素**を中央にする（`align-items` は上下の中央寄せ）

- 例：ボタン自体を左右中央に置きたい → ボタンに `margin` 左右 `auto`
- 例：ボタンの中の文字を左右中央にしたい → ボタンに `display: flex;` と `justify-content: center;`
- `margin` 左右 `auto` で中央寄せできる条件（第3回の復習）
    1. 表示形式がブロックであること
    2. `width` か `max-width` が指定されていること
- それぞれ使える条件があるので、CSSで中央寄せする方法まとめ で確認する

- 中央寄せの参考記事（note）

応用：中央だけじゃない「基準点からのずらし方」

`transform: translate()`の値は、X軸・Y軸それぞれ独立して指定できる。「その軸を中央寄せしたいときだけ`-50%`」「基準点をそのまま端として使いたいときは`0%`」「反対側の端に揃えたいときは`-100%`」という考え方で組み合わせる。

例えばハンバーガーメニューの3本線のように、X軸は常に`left: 50%; transform: translateX(-50%);`で横方向だけ中央寄せしつつ、Y軸は`top`の値（`0%`／`50%`／`100%`）と`translateY`の値（`0`／`-50%`／`-100%`）を線ごとに変えることで、上端・中央・下端に配置し分けることができる。

揃えたい位置	top（またはleft）	transform
上端（or左端）に揃える	0%	0%
中央に揃える	50%	-50%
下端（or右端）に揃える	100%	-100%

縦横どちらも中央にしたい場合は、この表のY軸・X軸をそれぞれ「中央に揃える」の組み合わせ（`top:50; left:50%; transform: translate(-50%, -50%);`）にすればよい。片方の軸だけ中央寄せしたい場合は、もう片方の軸の`transform`を`0%`のままにしておく。

装飾ラインを左右対称に配置する（疑似要素の応用）

見出しの左右に短いラインを入れるような装飾は、`::before`と`::after`を組み合わせて作る。

```
.sample-title {
  position: relative;
}

.sample-title::before,
.sample-title::after {
  content: "";
  position: absolute;
  top: 50%;
  transform: translateY(-50%);
  height: 2px;
  width: 24px;
  background: var(--primary-color);
}
```

```
.sample-title::before {
  left: 0;
}

.sample-title::after {
  right: 0;
}
```

- `::before`と`::after`をカンマでまとめて共通スタイル（線の太さ・色など）を指定し、位置（`left`/`right`）だけを個別に上書きすることで、左右対称の装飾を効率よく作れる

吹き出し・三角形のデザイン

擬似要素（`::before`）を使って三角形を作り、吹き出しのように見せるテクニック。

```
.sample_speech {
  position: relative;
}

.sample_speech::before {
  content: "";
  position: absolute;
  top: 100%;
  left: 50%;
  transform: translateX(-50%);
  border-width: 32px;
  border-style: solid;
  border-color: var(--primary-color) transparent transparent transparent;
}
```

- 三角形は装飾なので、HTMLタグを増やさず擬似要素で作る
- 三角形は、`width`と`height`を0にした要素に対して、4辺のボーダーの色を「表示したい方向だけ色をつけ、残りは`transparent`（透明）にする」ことで作られている
  - 原理が難しければ 三角形ジェネレーターを使えばよい
  - 最近は`clip-path`で図形を作る人も増えている（ clip-pathで三角形を作る）
- `border-color: var(--primary-color) transparent transparent transparent;`（上だけ色がついている）ため、要素の下側に下向きの三角形が表示される

- `top: 100%;` でセクションのすぐ下に配置し、`left: 50%; transform: translateX(-50%);` で横方向中央に配置している

別パターン：テキストを囲む吹き出し

```
.balloon_container {
  position: relative;
  display: inline-block;
  background: #ff9158;
  border-radius: 30px;
  padding: 16px;
  font-weight: bold;
  color: #fff;
}

.balloon_container::before {
  content: '';
  border-width: 10px;
  border-style: solid;
  border-color: #ff9158 transparent transparent transparent;
  position: absolute;
  top: 100%;
  left: 50%;
  transform: translateX(-50%);
}
```

linear-gradient()（グラデーション）

背景を単色ではなく、2色以上の色を滑らかに変化させて表示できる関数。

```
background: linear-gradient(方向, 色1, 色2, ...);
```

```
.gradient_item {
  background: linear-gradient(to right, #ffbb1e, #ff9158);
}
```

- `to right`：左から右方向へグラデーションする（`to left`、`to bottom`や、`45deg`のように角度で指定することも可能）

- 色はHEX（`#ff9b0f33`）でもRGBA（`rgba(255, 155, 15, 0.2)`）でも書ける。Figmaでは色の欄の「Hex」をクリックすると表記を切り替えられる（📎 カラーコードの表記の違い）
- ショートハンドで1行に書いても、`background-image`などに分けて書いてもよい（好みでOK）
- 📎 linear-gradientでしましま模様を作る（応用）

色々な影の表現

- 📎 box-shadowだけじゃない！CSSの色々な影の表現

ハンバーガーメニュー（基本形）

３本線のアイコンのような形をしていて、クリックするとナビゲーションが開ける、慣例的な機能的デザインの一つ。

```
<button class="hamburger">
  <span class="hamburger_bar"></span>
  <span class="hamburger_bar"></span>
  <span class="hamburger_bar"></span>
</button>
```

```
.hamburger {
  width: 40px;
  height: 32px;
  position: relative;
}

.hamburger_bar {
  width: 100%;
  height: 4px;
  position: absolute;
  left: 0;
  background: #000;
}

.hamburger_bar:nth-child(1) {
  top: 0;
}
```

```
.hamburger_bar:nth-child(2) {
  top: 50%;
  transform: translateY(-50%);
}

.hamburger_bar:nth-child(3) {
  top: 100%;
  transform: translateY(-100%);
}
```

- `position`：これを使って配置することで、クリックした時の変形（アニメーション）をやりやすくしている
- `spanタグ`：インライン要素なので、`height`をつけるときは必ず`display: block;`（または`position: absolute;`）をかけること
    - `position: absolute;`をつけると幅と高さの指定が効くようになる（ position:absoluteをつけた時の性質まとめ）

buttonタグを使う理由

比べるタグ	buttonを使う理由
`a`	`a`はクリックで別ページに移動するもの。ハンバーガーメニューはメニューが開閉するだけなので`button`
`div`	`button`はアクセシビリティが高い。キーボード（Tabキー）で操作でき、読み上げソフトがボタンとして認識する

- 3本線を`button`で囲い、1本1本の線は`span`で表す（ほぼテンプレとして覚えてよい）
- ハンバーガーメニューを`m_`にするのは、ヘッダーの外に置くデザインになる可能性もあるため（`l_`という解釈もある）
- 見た目のアレンジは様々なので丸暗記は不要。`position`と`transform`の位置調整を理解しておく
- ハンバーガーメニュー実装サンプル集／ハンバーガーメニューの事例と最適化のコツ（卒業後）

CSSの流れ 1. ハンバーガーメニュー全体の大きさを指定する（`width`/`height`） 2. 横長の細長い長方形を3つ作る（`width`/`height`） 3. 3つの長方形に背景色を当てる（`background`） 4. 等間隔に配置する（`position`＋位置調整）

- 定番の作り方なので、スニペット登録やコードのストックをしておくとよい
- うまくいかないときは、検証ツールで確認しながら進める

text-shadow

テキスト要素に対して、影をつけることができるCSSプロパティ。

```
text-shadow: X軸方向の距離 Y軸方向の距離 ぼかしの大きさ 色;
```

```
.txt-shadow_item {
  text-shadow: 8px 8px 3px #ccc;
}
```

- `box-shadow`は要素（箱）に対して影をつけるが、`text-shadow`は文字そのものに対して影をつける点が異なる

background-positionとbackground-sizeプロパティ

背景画像の指定（`url(...) center top / cover`のような書き方）は、実は複数のプロパティをまとめて書ける省略記法（ショートハンド）になっている。

```
.sample {
  background-image: url(../img/bg.jpg);
  background-position: center top;
  background-size: cover;
}
/* 上記3行をまとめて書いたものが↓ */
.sample {
  background: url(../img/bg.jpg) center top / cover;
}
```

- `background-image`：表示する画像（や`linear-gradient()`）を指定する
- `background-position`：画像の表示位置を指定する（`center top`＝横中央・縦上）
- `background-size`：画像の大きさ・トリミング方法を指定する（`cover`＝要素を隙間なく覆うように拡大縮小）

`background-image`と`background-size`は別々の役割を持つプロパティなので、実際は同時に使うのが普通（`background-size`だけを単独で使っても、表示する画像がなければ意味がない点に注意）。

- `background`は、`background-color`・`background-image`などを1行にまとめて指定できるショートハンド
- `background`を2回書くと、後に書いた方で上書きされ、前の指定は効かなくなるので注意

```
.sample {
  background: #fff;              /* ← 次の行で上書きされて効かない */
  background: url(../img/bg.jpg);
}
```

- backgroundのショートハンドの書き方

相対パスの考え方（フォルダをまたいだ参照）

CSSファイルの中から画像を参照するとき（`url(../img/bg_kv.jpg)`など）は、CSSファイル自身の場所を基準にした相対パスで書く。

```
src/
├ index.html
├ css/
│   ├ style.css   ← ここから見たパスを考える
│   └ reset.css
└ img/
    └ logo.png
```

- `css`フォルダと`img`フォルダは、どちらも`src`フォルダの中にある「兄弟フォルダ」の関係
- `style.css`から見て、まず`../`で1つ上の階層（`src/`）に戻り、そこから`img/logo.png`と辿るので、正しいパスは`../img/logo.png`
- `img/logo.png`（`../`なし）だと「同じ階層に`img`フォルダがある」という意味になってしまい、`css`フォルダの中を探しにいってしまうので誤り
- `/img/logo.png`（先頭に`/`）は「サーバーの一番上の階層（ドメイン直下）からの絶対パス」という別の意味になる
- 相対パスと絶対パス

複数クラスで擬似要素の文字を切り替える

```
/* 共通のスタイル（サイズ・色・位置）は共通クラスに */
.m_section_title::before {
  font-size: 48px;
  color: rgba(255, 187, 30, 0.6);
  position: absolute;
  bottom: 4px;
```

```
  left: 50%;
  transform: translateX(-50%);
}

/* 違う部分（文言）はModifierのクラスに */
.m_section_title__about::before {
  content: "ABOUT US";
}
```

navタグの代わりのrole="navigation"

```
<div class="l_footer-nav" role="navigation">
```

- 1ページに複数のナビゲーションがあるとき、主要なものは`nav`、そうでないものは`role="navigation"`にすることがある
- スクリーンリーダーにナビゲーション領域だと伝えつつ、HTMLの構造的な意味は控えめにできる

position:absoluteの基準要素について（補足）

`position: absolute;`をかけた要素は、「`position`に`relative`・`absolute`・`fixed`のいずれかが指定された、一番近い祖先要素」を基準に配置される。

- 基準にしたい要素にわざわざ`position: relative;`をかけるのは「狙った場所を基準にするため」であり、必須ではない
- もし祖先要素の誰にも`position`が指定されていない場合、`absolute`の要素は一番外側（`html`要素＝画面全体）を基準に配置される
- そのため、「`absolute`を使うときは必ず`relative`の指定が必要」という説明は誤りで、正しくは「狙った基準にしたいときは`relative`（など）を指定する」という理解が正確

本日のまとめ

1. ハンバーガーメニューの見た目の作り方がわかった
2. `linear-gradient`でフィルターをかけることができる
3. `position`での中央寄せの手順がわかった
4. CSS変数・`l_`/`m_`接頭辞など、クラス名を意識したコーディングの実践方法がわかった

参考記事

- テキストに影をつけよう
- positionとtransformで要素を上下左右中央寄せしよう
- linear-gradient()を使ってグラデーションしてみよう
- CSSで吹き出しを作ろう
- 吹き出しを作ろう（授業ページのリンク）
- 複数背景で画像の上に半透明のレイヤーを載せる方法
- CSSで三角形を作る手順を初心者でも分かるように丁寧に解説！
- CSSでハンバーガーメニューアイコンを作ろう
- 中央寄せコツ：https://saruwakakun.com/html-css/basic/centering